

تقرير مشروع مادة البرمجة المتوازية

عمل الطلاب:

نيرمين شادي الظواهره

يارا شادي مرتك

جنى حنا العقلة

بطرس بطرس سلمون

ميّار جورج مصلح

تقرير مشروع مادة البرمجة المتوازية

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

High-Performance E-Commerce Backend Engine

ملخص تنفيذي

يقدم المشروع محركاً خلفياً لمنظومة تجارة إلكترونية مصممة للتعامل مع ضغط عالٍ وعدد كبير من طلبات إتمام الشراء المتزامنة. التركيز في المشروع ليس على كثرة الشاشات، بل على استقرار النظام عند الوصول المتزامن للبيانات، والتحكم بالسعة، والمعالجة الخلفية، والمعالجة على دفعات، وتوزيع الحمل.

تم بناء النظام باستخدام إطار عمل لارافيل مع تقسيم واضح بين طبقة التحكم، طبقة الخدمات، الأعمال الخلفية، والطبقة الوسيطة. هذا التقسيم جعل المتطلبات غير الوظيفية قابلة للتنفيذ والاختبار بشكل مستقل بدل أن تكون متداخلاً داخل المتحكمات.

التصميم المعماري العام

يعتمد المشروع على بنية طبقية. يبدأ الطلب من طبقة المسارات، ثم يمر عبر الطبقة الوسيطة الخاصة بتوزيع الحمل والتحكم بالسعة والصلاحيات، وبعدها يصل إلى طبقة التحكم التي تفوض منطق العمل إلى طبقة الخدمات. أما الأعمال التي لا يحتاج المستخدم إلى انتظارها فتنتقل إلى صفوف العمل الخلفية.

الطبقة	المسؤولية داخل المشروع
Client / API Consumer	يرسل طلبات API مثل عرض المنتجات، إنشاء طلب، أو تنفيذ Checkout
Routes	توجيه الطلب إلى Controller المناسب وتطبيق الـ Middleware.
Middleware	توزيع الحمل، التحكم بعدد العمليات المتوازية، الصلاحيات، وتسجيل الأداء.
Controllers	استقبال الطلب والتحقق من الاستجابة بدون وضع منطق ثقيل.
Services	تنفيذ منطق العمل مثل Checkout، الدفع، المخزون، التقارير، والسعة.
Jobs / Queues	تنفيذ الأعمال غير المتزامنة مثل الفواتير والإشعارات وتقارير المبيعات.
Models / Database	تمثيل الجداول، المعاملات، الأقفال، وسجلات التتبع.

المتطلبات الوظيفية وكيف تم توظيفها

المتطلبات الوظيفية في المشروع لم توضع كوظائف متجر فقط، بل استخدمت كسيناريوهات عملية لاختبار المتطلبات غير الوظيفية. لذلك تم اختيار عمليات مثل إتمام الشراء والدفع وإدارة المخزون لأنها أكثر نقاط النظام حساسية تحت الضغط.

المتطلب الوظيفي	توظيفه لخدمة المتطلبات غير الوظيفية
تسجيل المستخدم وتسجيل الدخول	تحديد صاحب الطلب والمحفظه، وربط السجلات بالمستخدم.
عرض المنتجات وإدارة المخزون	توفير مورد مشترك قابل للتنافس بين عدة مستخدمين.
السلة وعناصر الطلب	تحويل المنتج والكمية إلى أسطر واضحة يمكن معالجتها وقفلها.
Checkout والدفع	السيناريو الأساسي لاختبار Atomicity و Race Condition و Idempotency.
الفواتير والإشعارات	أعمال تابعة للدفع تم نقلها إلى Queue لتقليل زمن الاستجابة.
حركات المخزون	مصدر بيانات تشغيلي لتتبع stock_before و stock_after وبناء التقارير.
تقارير المبيعات اليومية	اختبار Batch Processing عبر معالجة حركات البيع على دفعات.
العقد والخوادم وسجلات الطلبات	تمثيل Load Balancing ومراقبة الأداء والسعة.

الخوارزميات والأنماط المستخدمة

الخوارزمية / النمط	مكان الاستخدام	الهدف
Database Transaction	OrderCheckoutService	تنفيذ Checkout كوحدة ذرية: إما يكتمل كله أو يفشل كله.
Pessimistic Row-Level Locking lockForUpdate	الطلب والمنتجات أثناء Checkout	منع تعديل نفس المخزون من أكثر من Request بنفس اللحظة.
Stable Lock Ordering	قفل المنتجات بترتيب ثابت حسب id	تقليل احتمال حدوث Deadlock عند قفل أكثر من منتج.
Idempotency Key	payments.idempotency_key	منع تكرار الدفع لنفس العملية عند إعادة إرسال الطلب.
+ Cache Atomic Lock Reservation Tokens	CapacityService	ضبط عدد العمليات المتوازية ضمن حد معين.

نقل الأعمال غير الضرورية للمسار الخلفى.	Notification Jobs و Invoice	Asynchronous Queue
قراءة حركات البيع على دفعات لتقليل استهلاك الذاكرة.	DailySalesReportBatchService	Chunked Batch / Processing chunkById
اختيار الخادم الأقل حملاً مع كسر التعادل عشوائياً.	LoadBalancerService	Least-Loaded Random Tie Load Balancing
تسجيل الأداء كسلوك جانبي دون تلويث منطق العمل.	Request Logging و Performance Monitoring	AOP via Middleware

المتطلبات غير الوظيفية وتصميم قاعدة البيانات التشغيلي

كل مطلب في المشروع قابل للتنفيذ والاختبار بشكل نظيف. الفكرة الأساسية كانت نقل التصميم من مخطط عام إلى مخطط تشغيلي: جداول أساسية للتجارة، جداول لتتبع المخزون، جداول للأعمال الخلفية، وجدول للبنية التحتية.

حماية البيانات المشتركة من التضارب

المشكلة المعالجة هنا هي منع بيع كمية أكبر من المتوفر عند تنفيذ عدة عمليات إتمام شراء في الوقت نفسه. تم حل المشكلة باستخدام معاملة قاعدة بيانات مع قفل تشاؤمي على صفوف الطلب والمنتجات، ثم التحقق من المخزون قبل الخصم.

دور مخطط قاعدة البيانات: فصل الكيانات الحساسة جعل التحكم أدق. فكل منتج في السلة أو الطلب أصبح صفاً مستقلاً، وسجل حركة المخزون يحتفظ بالكمية قبل وبعد العملية مع ربطها بالطلب والمنتج. هذا أعطى صفوفاً واضحة يمكن قفلها وتتبعها أثناء الدفع. كما أن مفتاح منع تكرار الدفع ساعد على منع تنفيذ العملية نفسها أكثر من مرة.

- الخوارزمية المستخدمة: قفل تشاؤمي على مستوى الصف مع معاملة قاعدة بيانات.
- النتيجة المتوقعة: نجاح الطلبات التي يغطيها المخزون، وفشل الطلبات الزائدة برسالة تعارض صحيحة دون فساد في البيانات.
- معيار الاختبار: عدم نزول المخزون تحت الصفر، وعدم إنشاء دفعات أو فواتير للطلبات الفاشلة.

إدارة الموارد والتحكم بالسعة

المشكلة المعالجة هنا هي منع النظام من استهلاك موارده بشكل مفرط عند الضغط. تم تحديد حد أعلى للعمليات المتزامنة لكل مجموعة من العمليات الحساسة، وعند امتلاء السعة يرجع النظام رفضاً مؤقتاً، ثم يحرر الحجز بعد انتهاء الطلب.

- دور البنية التشغيلية في المشروع: أضيفت جداول للخوادم وسجلات الطلبات ونتائج القياس لدعم المراقبة العامة للنظام، بينما حالة السعة نفسها تُدار عبر طبقة cache والقفل الذري.
- الخوارزمية المستخدمة: الخوارزمية المستخدمة: قفل ذري عبر طبقة cache مع reservation tokens لإدارة السعة المتاحة لكل مجموعة عمليات.
- النتيجة المتوقعة: قبول العمليات ضمن الحد المسموح، ورفض العمليات الزائدة مؤقتاً برسالة واضحة.

- معيار الاختبار: عند امتلاء السعة يرجع النظام استجابة رفض مؤقتة، وبعد انتهاء عملية إتمام الشراء تصبح السعة متاحة من جديد.

المعالجة غير المتزامنة

المشكلة المعالجة هنا هي أن المستخدم لا يجب أن ينتظر الأعمال التابعة للدفع مثل إصدار الفاتورة وإرسال الإشعار. بعد نجاح الدفع يتم إنشاء سجل الدفع فوراً، ثم ترسل مهام الفاتورة والإشعار إلى صف العمل الخلفي لتنفيذ لاحقاً.

دور مخطط قاعدة البيانات: فصلت الجداول بين نتيجة الدفع الأساسية وبين الآثار التابعة للدفع، مثل الفواتير والإشعارات. هذا الفصل سمح بإنهاء عملية إتمام الشراء بسرعة، ثم تنفيذ الأعمال التابعة في الخلفية دون خلطها مع المعاملة الأساسية.

- النمط المستخدم: نمط المنتج والمستهلك.
- النتيجة المتوقعة: يتم تسجيل الدفع مباشرة، بينما تبقى مهام الفاتورة والإشعار في صف العمل إلى حين تشغيل العامل الخلفي.
- معيار التحقق: عند استخدام database queue وإيقاف العامل الخلفي، يُنشأ الدفع أولاً وتبقى الأعمال التابعة مؤجلة. وبعد تشغيل العامل الخلفي تُنشأ سجلات الفاتورة والإشعار.

المعالجة على دفعات

المشكلة المعالجة هنا هي توليد تقرير يومي من عدد كبير من حركات البيع دون تحميل كل البيانات في الذاكرة. لذلك تتم قراءة حركات المخزون على دفعات صغيرة، ثم تجمع النتائج في جداول التقارير اليومية.

دور مخطط قاعدة البيانات: وجود سجل تفصيلي لحركات المخزون جعل بناء التقارير اليومية ممكناً ونظيفاً. كما أن وجود جداول للتقرير النهائي وفر مكاناً منظماً لحفظ نتائج التجميع.

- الخوارزمية المستخدمة: معالجة على دفعات باستخدام المفتاح المتزايد.
- النتيجة المتوقعة: توليد تقرير صحيح حتى عند وجود حجم كبير من البيانات.
- معيار الاختبار: عند تقليل حجم الدفعة إلى 2، يبقى التقرير النهائي صحيحاً وتنجح كل التحقيقات.

توزيع الأحمال

المشكلة المعالجة هنا هي محاكاة توزيع الطلبات على أكثر من خادم. يختار النظام الخادم الأقل حملاً نسبة إلى سعته القصوى، ويتجاهل الخوادم غير النشطة، ثم يسجل نتيجة الاختيار مع الطلب.

دور مخطط قاعدة البيانات: وجود جدول للخوادم وجدول لسجلات الطلبات جعل توزيع الحمل قابلاً للقياس. صار لكل خادم حالة وسعة وحمل حالي، وصار لكل طلب سجل يوضح الخادم الذي عالجه.

- الخوارزمية المستخدمة: تفضيل العقدة النشطة أولاً، ثم اختيار العقدة الأقل حملاً نسبة إلى سعتها، ثم الأقل current_load، مع كسر التعادل عشوائياً.
- النتيجة المتوقعة: اختيار الخادم الأنسب وفق الحمل الحالي، وإظهار معلومات العقدة المختارة وترويسة استراتيجية التوزيع في الاستجابة.
- معيار الاختبار: التحقق من العقدة المختارة عبر response headers، وتسجيل الطلب في request_logs مع server_node_id و operation_name و status_code.

مراقبة الأداء باستخدام AOP

تم تطبيق مفهوم cross-cutting concerns عبر Middleware لمحاكاة جانب AOP في مراقبة الأداء وتسجيل الطلبات دون خلطها مع منطق الأعمال. لذلك يتم قياس زمن الاستجابة وتسجيل العملية تلقائياً حول الطلب الأساسي دون تلويث منطق العمل.

جانب AOP	التطبيق في المشروع
Performance Monitoring	قياس response_time_ms وتخزينه في request_logs.
Load Balancing	تعيين server node لكل request عبر Middleware.
Capacity Control	رفض الطلبات الزائدة قبل الوصول إلى منطق العمل.
Clean Business Logic	بقاء OrderCheckoutService مركزاً على الدفع والمخزون فقط.

الاختبارات والنتائج العملية

اختبار مشكلة التزامن أثناء إتمام الشراء

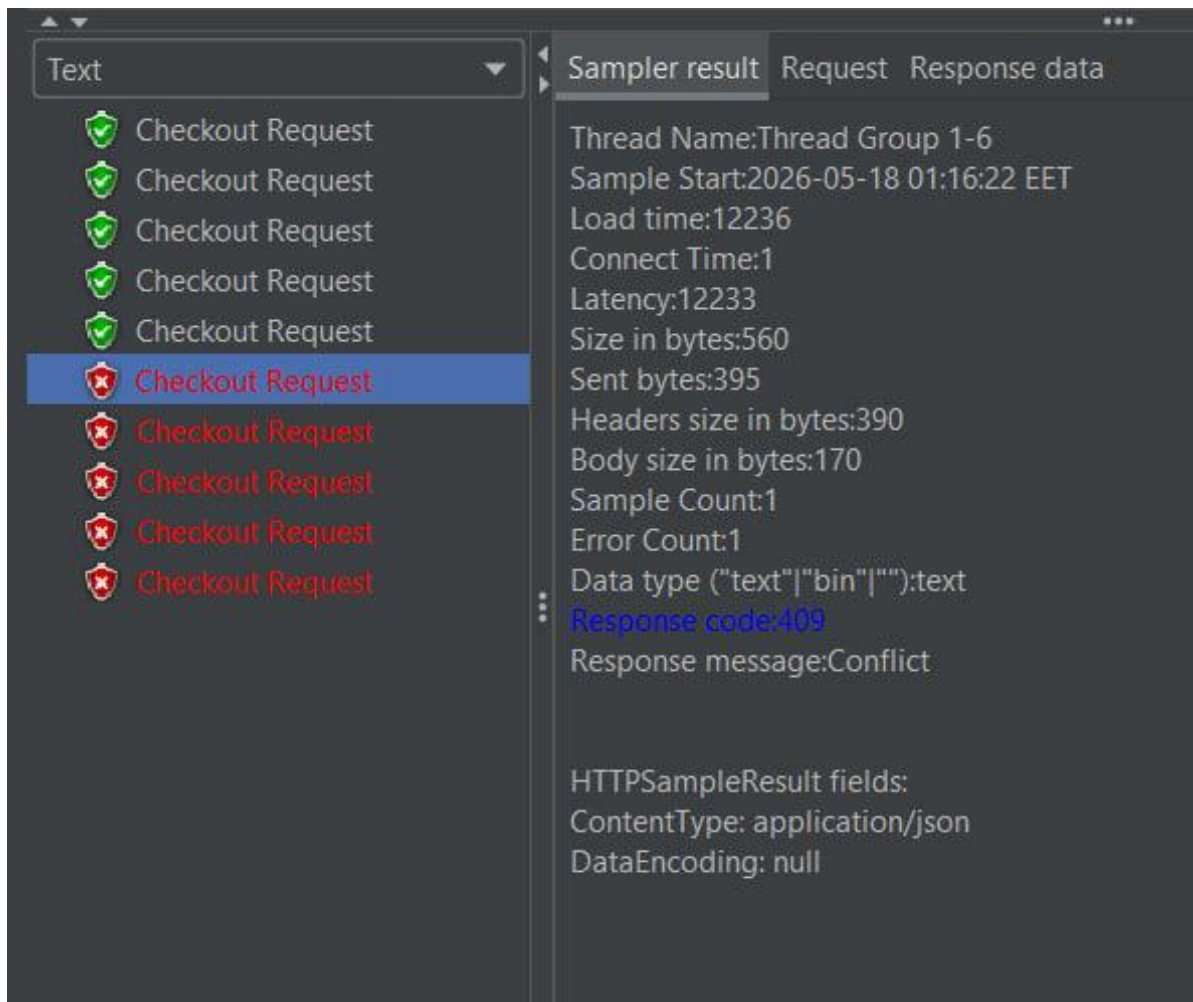
البند	القيمة
الهدف	إثبات عدم بيع كمية أكبر من المخزون عند وجود طلبات متزامنة.
السيناريو	منتج كميته 5، وتم إنشاء 10 طلبات pending/unpaid لنفس المنتج.
النتيجة	نجاح 5 عمليات Checkout وفشل 5 عمليات HTTP 409 Conflict
الدليل	المخزون أصبح 0، وتم إنشاء 5 payments و 5 invoices و 5 notifications فقط.

id	name	description	price	stock_quantity	quantity_counter	status	photos	created_at	updated_at
1	Postman Product 272082	Created from Postman collection	50.00	5	5	active	[]	2026-05-17 22:01:12	2026-05-17 22:01:12

حالة المنتج قبل التنفيذ: كمية المخزون خمس قطع وسعر القطعة خمسون.

id	user_id	balance	created_at	updated_at
1	2	1000.00	2026-05-17 22:05:42	2026-05-17 22:06:06

محفظة المستخدم قبل الدفع وإتمام الشراء.



تفصيل إحدى عمليات الفشل برسالة تعارض.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
Checkout R...	10	10006	2545	13365	3649.66	50.00%	42.1/min	0.66	0.27	959.0
TOTAL	10	10006	2545	13365	3649.66	50.00%	42.1/min	0.66	0.27	959.0

Summary Report من JMeter يوضح عدد الطلبات ونتائجها.

id	name	description	price	stock_quantity	quantity_counter	status	photos	created_at	updated_at
1	Postman Product 272082	Created from Postman collection	50.00	0	0	active	[]	2026-05-17 22:01:12	2026-05-17 22:16:34

حالة المنتج بعد التنفيذ: وصول الكمية إلى 0 بدون قيم سالبة.

id	order_id	payment_method	transaction_reference	idempotency_key	amount	status	paid_at	created_at	updated_at
1	1	wallet	PAY-1779056184-61XYI9RK	jmeter-9b15da0f-23e7-4b22-8dda-8ce2caea4dc9	50.00	completed	2026-05-17 22:16:24	2026-05-17 22:16:24	2026-05-17 22:16:24
2	2	wallet	PAY-1779056187-NI0730W7	jmeter-1e9faf8c-ea85-4c9c-bbba-630a0a92e14e	50.00	completed	2026-05-17 22:16:27	2026-05-17 22:16:27	2026-05-17 22:16:27
3	3	wallet	PAY-1779056189-LCYKIGHM	jmeter-c95a8d2b-09f0-4c02-8da0-c66de76dbb47	50.00	completed	2026-05-17 22:16:29	2026-05-17 22:16:29	2026-05-17 22:16:29
4	4	wallet	PAY-1779056192-TG0JKDLW	jmeter-b40c67ec-44ec-49b9-b464-f7366278e835	50.00	completed	2026-05-17 22:16:32	2026-05-17 22:16:32	2026-05-17 22:16:32
5	5	wallet	PAY-1779056194-ZTJO6NQ6	jmeter-bbdba4cb-6d49-4adb-970f-04c2c71d2aaa	50.00	completed	2026-05-17 22:16:34	2026-05-17 22:16:34	2026-05-17 22:16:34

جدول payments بعد الاختبار: إنشاء 5 عمليات دفع فقط.

id	order_id	invoice_number	status	issued_at	created_at	updated_at
1	1	INV-20260517222611-OZTSQPAV	issued	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
2	2	INV-20260517222611-YVQF9ZCF	issued	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
3	3	INV-20260517222611-FK7BOS0Z	issued	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
4	4	INV-20260517222611-MPZDD9EC	issued	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
5	5	INV-20260517222612-YOVZPQRN	issued	2026-05-17 22:26:12	2026-05-17 22:26:12	2026-05-17 22:26:12

جدول invoices بعد الاختبار: إنشاء 5 فواتير للطلبات الناجحة فقط.

id	user_id	order_id	type	message	status	sent_at	created_at	updated_at
1	2	1	payment_success	Payment for order ORD-1779055729 was completed suc...	sent	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
2	2	2	payment_success	Payment for order ORD-1779055908 was completed suc...	sent	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
3	2	3	payment_success	Payment for order ORD-1779055915 was completed suc...	sent	2026-05-17 22:26:11	2026-05-17 22:26:11	2026-05-17 22:26:11
4	2	4	payment_success	Payment for order ORD-1779055922 was completed suc...	sent	2026-05-17 22:26:12	2026-05-17 22:26:12	2026-05-17 22:26:12
5	2	5	payment_success	Payment for order ORD-1779055927 was completed suc...	sent	2026-05-17 22:26:12	2026-05-17 22:26:12	2026-05-17 22:26:12

جدول notifications بعد الاختبار: إنشاء 5 إشعارات للطلبات الناجحة فقط.

id	user_id	balance	created_at	updated_at
1	2	750.00	2026-05-17 22:05:42	2026-05-17 22:16:34

محفظة المستخدم بعد إتمام عمليات الشراء الناجحة.

الخلاصة: نجح النظام في منع مشكلة التزامن لأن الطلبات التي وصلت بعد نفاذ المخزون فشلت بشكل صحيح دون إنشاء دفعات أو حركات مخزون إضافية، ودون أن يصبح المخزون أقل من الصفر.

اختبار المعالجة غير المتزامنة

البند	القيمة
الهدف	إثبات أن إصدار الفاتورة والإشعار لا ينفذان داخل مسار Checkout الرئيسي.
السيناريو	إيقاف Queue Worker، تنفيذ Checkout، ثم فحص payments و invoices و jobs و notifications.
قبل تشغيل Worker	تم إنشاء payment، وبقيت invoices و notifications فارغة، وظهر Job ان في jobs.
بعد تشغيل Worker	أصبح jobs فارغاً وتم إنشاء invoice و notification.

id	order_id	payment_method	transaction_reference	idempotency_key	amount	status	paid_at	created_at	updated_at
1	1	card	PAY-1779064315-C7NORF2E	idem-1779064313	50.00	completed	2026-05-18 00:31:55	2026-05-18 00:31:55	2026-05-18 00:31:55

بعد إتمام الشراء: يظهر سجل الدفع في جدول الدفعات.

id order_id invoice_number status issued_at created_at updated_at

قبل تشغيل العامل الخلفي: جدول الفواتير فارغ.

id user_id order_id type message status sent_at created_at updated_at

قبل تشغيل العامل الخلفي: جدول الإشعارات فارغ.

id	queue	payload	attempts	reserved_at	available_at	created_at
1	default	{"uuid":"b76672fd-5d80-47ca-a6f5-c113e893bcda","di...	0	NULL	1779065057	1779065057
2	default	{"uuid":"8d70a29b-94a8-472b-a506-7eea88e629e4","di...	0	NULL	1779065057	1779065057

جدول المهام يحتوي مهمتين: فاتورة وإشعار.

id queue payload attempts reserved_at available_at created_at

بعد تشغيل العامل الخلفي: جدول المهام أصبح فارغاً.

id	user_id	order_id	type	message	status	sent_at	created_at	updated_at
1	2	1	payment_success	Payment for order ORD-1779064137 was completed suc...	sent	2026-05-18 00:35:57	2026-05-18 00:35:57	2026-05-18 00:35:57

بعد تشغيل العامل الخلفي: تم إنشاء الإشعار.

id	order_id	invoice_number	status	issued_at	created_at	updated_at
1	1	INV-20260518003557-WZXMQT2I	issued	2026-05-18 00:35:57	2026-05-18 00:35:57	2026-05-18 00:35:57

بعد تشغيل العامل الخلفي: تم إنشاء الفاتورة.

الخلاصة: تم فصل الأعمال التابعة للدفع عن المسار الرئيسي. المستخدم يحصل على استجابة أسرع، بينما تتم معالجة الفاتورة والإشعار لاحقاً بواسطة العامل الخلفي.

اختبار معالجة تقرير المبيعات على دفعات

البنء	القيمة
الهدف	إثبات أن تقرير المبيعات يعالج حركات المخزون على دفعات.
السيناريو	تقليل حجم الدفعة إلى 2، إنشاء عدة Inventory Movements، ثم تشغيل GenerateDailySalesReportJob.
النتيجة	نجاح 3 اختبارات و 17 assertion، وتوليد التقرير النهائي بشكل صحيح.

```
php artisan test tests/Feature/DailySalesReportBatchProcessingTest.php
```

```
PASS Tests\Feature\DailySalesReportBatchProcessingTest
```

```
generate endpoint dispatches background job ✓
```

```
artisan command dispatches daily sales report job ✓
```

```
job generates daily sales report in chunks from sale movements ✓
```

```
Tests: 3 passed (17 assertions)
```

```
Duration: 1.80s
```

الخلاصة: استخدام المعالجة على دفعات يثبت أن القراءة لا تتم دفعة واحدة. نجاح الاختبار بعد تخفيض حجم الدفعة يثبت أن التجميع النهائي صحيح حتى عندما تقسم البيانات إلى دفعات صغيرة.

اختبار التحكم بالسعة

الاختبار	ما يثبتته
رفض العملية عند امتلاء critical-operations	عند حجز السعة كاملة يرجع النظام 503 مع Retry-After بدلاً من تنفيذ عملية إضافية.
تحرير slot بعد انتهاء Checkout	بعد إكمال الطلب يصبح active = 0 وتعود السعة المتاحة إلى القيمة الصحيحة.

الخلاصة: التحكم بالسعة يمنع الانهيار تحت الضغط لأنه لا يسمح بعدد غير محدود من العمليات الحرجة. كما أن تحرير الحجز بعد الطلب يمنع تسرب الموارد.

اختبار توزيع الأحمال

الاختبار	ما يثبتته
اختيار أقل Node حملاً	عند وجود Node A بحمل 3 و Node B بحمل 1، يختار النظام Node B ويضيف headers توضح ذلك.
تسجيل الطلب في request_logs	يتم تسجيل user_id و server_node_id و operation_name و status_code و endpoint.
العمل بدون nodes	عند عدم وجود server nodes يستمر الطلب ويظهر X-Server-Node = unassigned.

نتيجة اختبار توزيع الأحمال:

توضح نتيجة الاختبار أن التوزيع كان متقارباً بين الخوادم الثلاثة. استقبل الخادم الثاني 634 طلباً، والخادم الأول 627 طلباً، والخادم الثالث 622 طلباً. تُظهر تجربة الضغط تقارباً عملياً في التوزيع بين الخوادم، وهو ما يشير إلى أن الحمل لم يُوجَّه إلى خادم واحد فقط.

الخلاصة: توزيع الحمل في المشروع ليس شكلياً، لأنه يعتمد على بيانات فعلية للخوادم ويسجل النتيجة في سجل الطلبات، مما يجعله قابلاً للمراقبة والاختبار.

```

Illuminate\Support\Collection {#6663
  all: [
    {#6657
      +"name": "server-2",
      +"server_node_id": 2,
      +"total": 634,
    },
    {#6659
      +"name": "server-1",
      +"server_node_id": 1,
      +"total": 627,
    },
    {#6661
      +"name": "server-3",
      +"server_node_id": 3,
      +"total": 622,
    },
  ],
}

```

والتالي صور توضح توزيع العمليات على اكثر من node (A,B,C) بتجربة على Jmeter:

The screenshot shows the JMeter interface with the 'Response headers' tab selected. The left pane lists 15 'HTTP Request' samplers, each with a green checkmark. The right pane displays the response headers for the selected request:

- 1 HTTP/1.1 200 OK
- 2 Host: 127.0.0.1:8000
- 3 Connection: close
- 4 X-Powered-By: PHP/8.3.31
- 5 Cache-Control: no-cache, private
- 6 Date: Mon, 18 May 2026 14:40:50 GMT
- 7 Content-Type: application/json
- 8 X-Capacity-Group: critical-operations
- 9 X-Capacity-Limit: 12
- 10 X-Capacity-Active: 0
- 11 X-Capacity-Remaining: 12
- 12 X-Load-Balancer-Strategy: least-loaded
- 13 X-Server-Node-Id: 1
- 14 X-Server-Node: Node A
- 15 X-Server-Host: 10.0.0.1
- 16 X-Server-Current-Load: 1
- 17 Access-Control-Allow-Origin: *
- 18

Text

- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request

Sampler result Request Response data

Response Body Response headers

```
1 HTTP/1.1 200 OK
2 Host: 127.0.0.1:8000
3 Connection: close
4 X-Powered-By: PHP/8.3.31
5 Cache-Control: no-cache, private
6 Date: Mon, 18 May 2026 14:40:53 GMT
7 Content-Type: application/json
8 X-Capacity-Group: critical-operations
9 X-Capacity-Limit: 12
10 X-Capacity-Active: 0
11 X-Capacity-Remaining: 12
12 X-Load-Balancer-Strategy: least-loaded
13 X-Server-Node-Id: 2
14 X-Server-Node: Node B
15 X-Server-Host: 10.0.0.2
16 X-Server-Current-Load: 1
17 Access-Control-Allow-Origin: *
```

Text

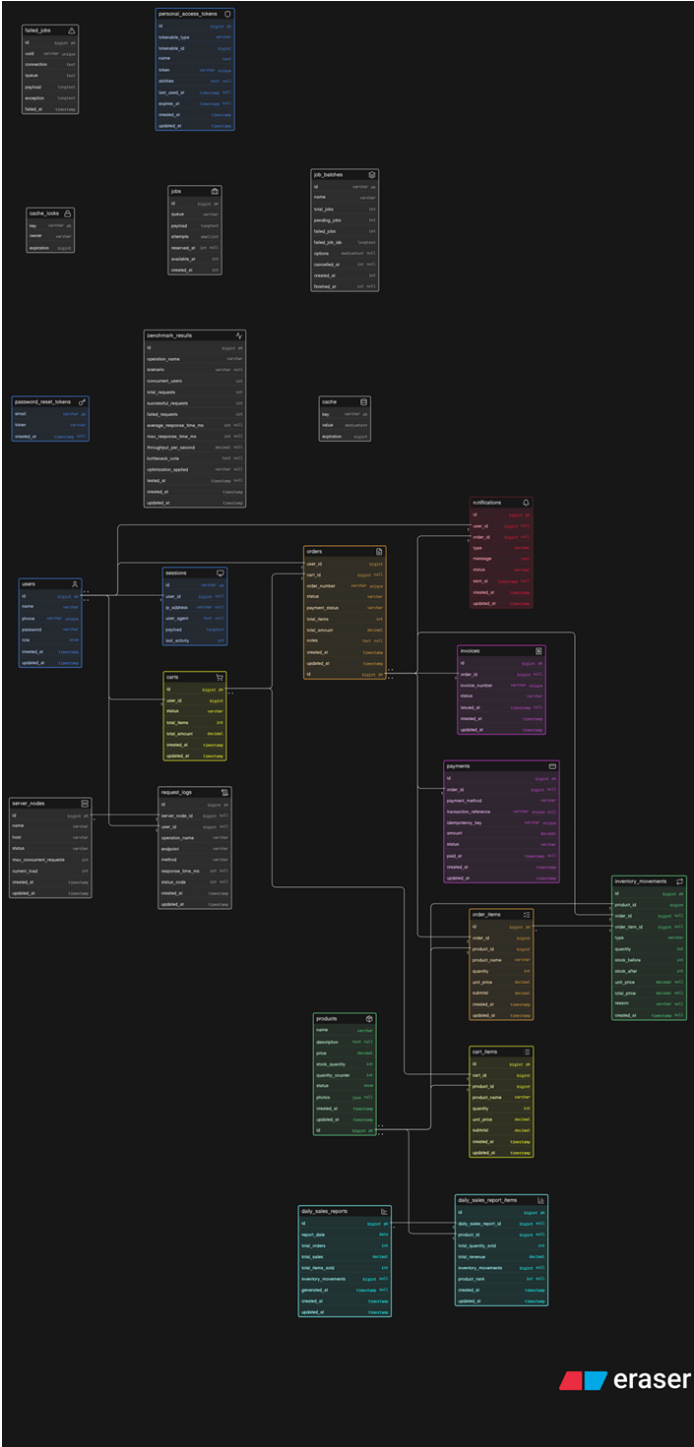
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request
- HTTP Request

Sampler result Request Response data

Response Body Response headers

```
1 HTTP/1.1 200 OK
2 Host: 127.0.0.1:8000
3 Connection: close
4 X-Powered-By: PHP/8.3.31
5 Cache-Control: no-cache, private
6 Date: Mon, 18 May 2026 14:41:32 GMT
7 Content-Type: application/json
8 X-Capacity-Group: critical-operations
9 X-Capacity-Limit: 12
10 X-Capacity-Active: 0
11 X-Capacity-Remaining: 12
12 X-Load-Balancer-Strategy: least-loaded
13 X-Server-Node-Id: 3
14 X-Server-Node: Node C
15 X-Server-Host: 10.0.0.3
16 X-Server-Current-Load: 1
17 Access-Control-Allow-Origin: *
18
```

ERD التشغيلي للمشروع



يوضح المخطط أن قاعدة البيانات لم تُصمم كتجارة إلكترونية فقط، بل ك ERD تشغيلي يدعم الاختبار، مراقبة الأداء، تتبع المخزون، الأعمال الخلفية، وتوزيع الحمل.

الخلاصة النهائية

يحقق المشروع هدف مادة البرمجة المتوازية لأنه يحول متجر إلكتروني بسيط إلى محرك خلفي قابل للاختبار تحت الضغط. تم استخدام عملية إتمام الشراء كسيناريو مركزي لمعالجة مشكلة التزامن وحماية المخزون، وتم دعم النظام بالتحكم بالسعة، صفوف العمل الخلفية، المعالجة على دفعات، وتوزيع الحمل، مع مراقبة الأداء عبر الـ `middleware`.

النتيجة النهائية أن كل متطلب غير وظيفي له تصميم بيانات واضح، خدمة مسؤولة عن التنفيذ، واختبار أو دليل عملي يثبت السلوك المتوقع.